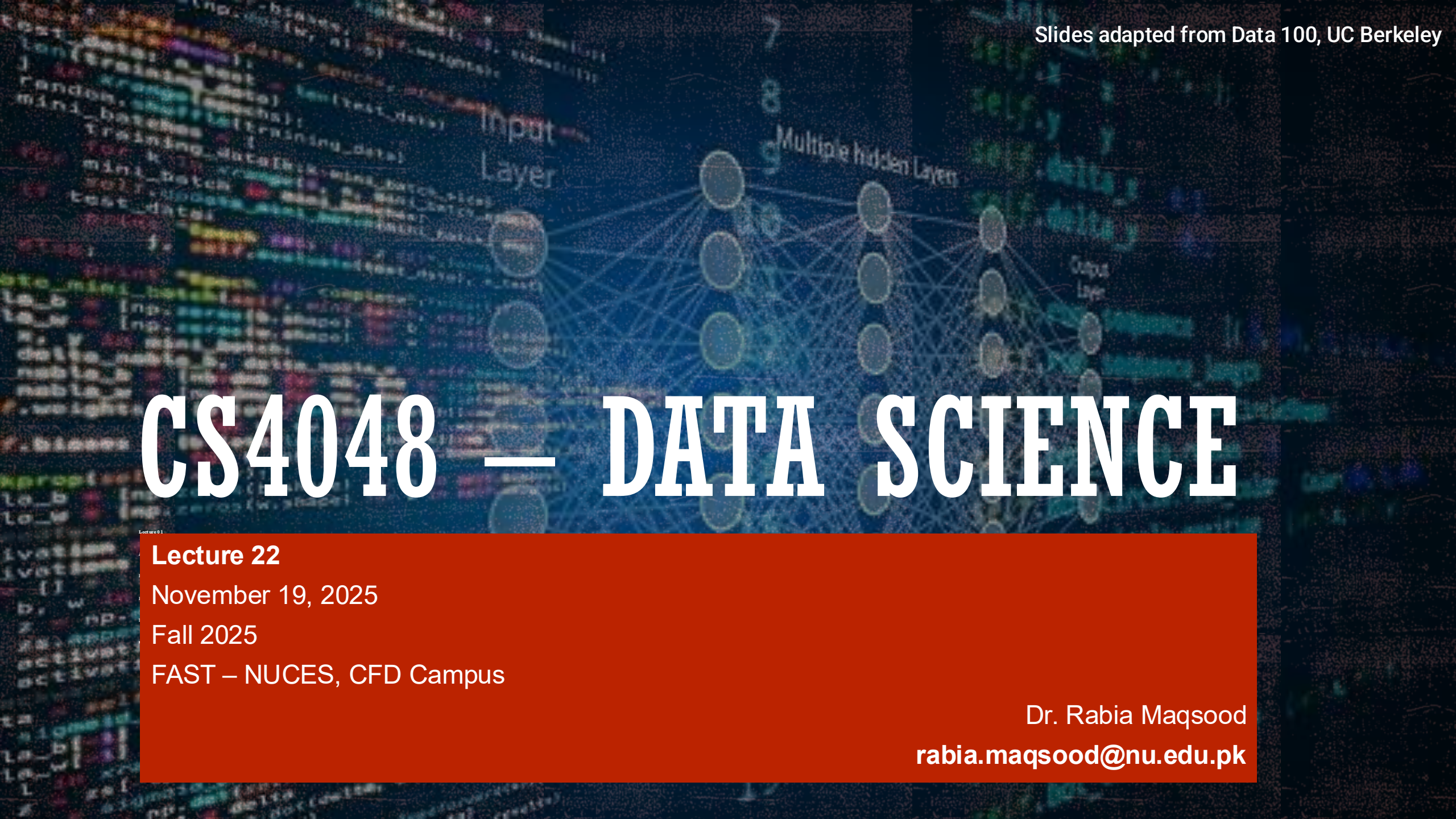


CS4048 — DATA SCIENCE



Lecture 22

November 19, 2025

Fall 2025

FAST – NUCES, CFD Campus

Dr. Rabia Maqsood

rabia.maqsood@nu.edu.pk

TODAY'S TOPICS

- Regression overview (python notebook)
- Feature Engineering + Regression
- Model overfitting/underfitting
- Model variance
- Data splitting
 - Train/test, validation sets
 - K-fold cross validation

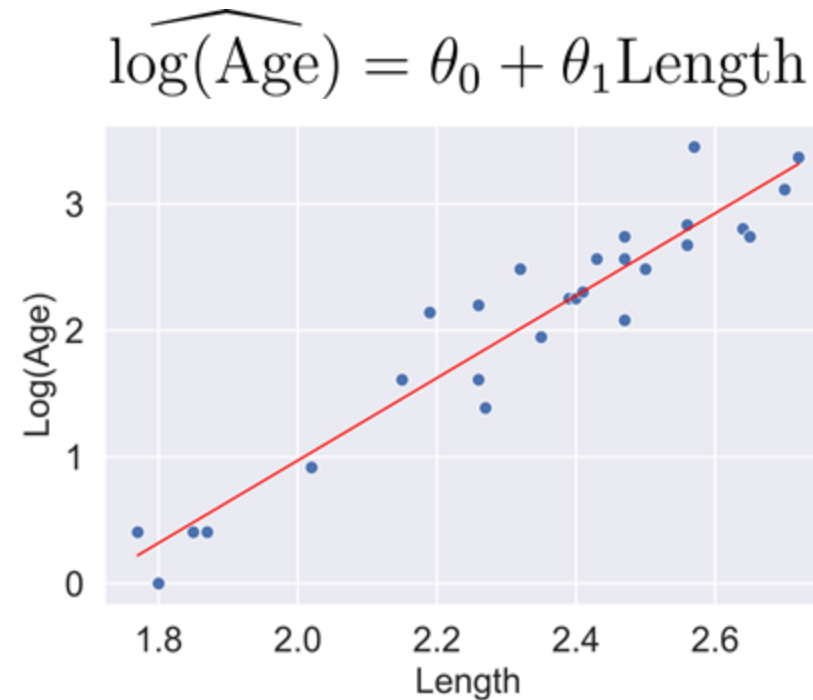
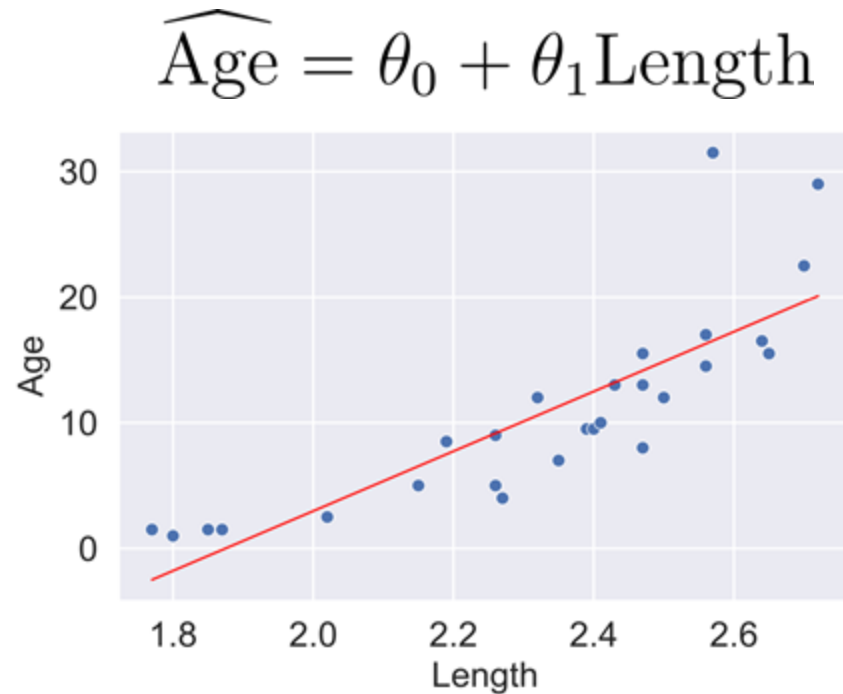


FEATURE ENGINEERING

TRANSFORMING FEATURES

Two observations from previous lectures:

- We found that applying a transformation could help **linearize** a dataset.
- We saw that **linear modeling works best** when our dataset has **linear relationships**.



FEATURE ENGINEERING

Feature engineering is the process of transforming raw features into more informative features for use in modeling.

Allows us to:

- Express non-linear relationships using linear models (e.g., $x \rightarrow \log(x)$)
- Use qualitative/categorical/non-numeric features in models (e.g., grade level)
- Capture domain knowledge (e.g., latitude+longitude $\rightarrow$ distance to nearest school)



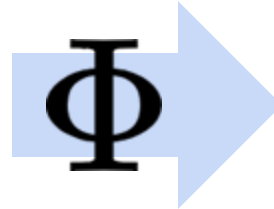
FEATURE FUNCTIONS

A **feature function** describes the transformation of raw features to transformed features.

Example: a feature function that adds a squared feature to the design matrix

	hp	mpg
0	130.00	18.00
1	165.00	15.00
2	150.00	18.00
...	...	...
395	84.00	32.00
396	79.00	28.00
397	82.00	31.00

392 rows × 2 columns



	hp	hp^2	mpg
0	130.00	16900.00	18.00
1	165.00	27225.00	15.00
2	150.00	22500.00	18.00
...	...	...	...
395	84.00	7056.00	32.00
396	79.00	6241.00	28.00
397	82.00	6724.00	31.00

392 rows × 3 columns

Dataset of raw
features: $\mathbb{X} \in \mathbb{R}^{n \times p}$

After applying the feature function Φ :

$$\Phi(\mathbb{X}) \in \mathbb{R}^{n \times p'}$$

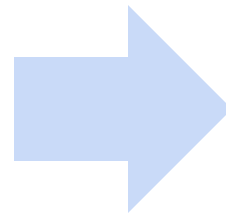


FEATURE FUNCTIONS

A **feature function** describes the transformation of raw features to transformed features.

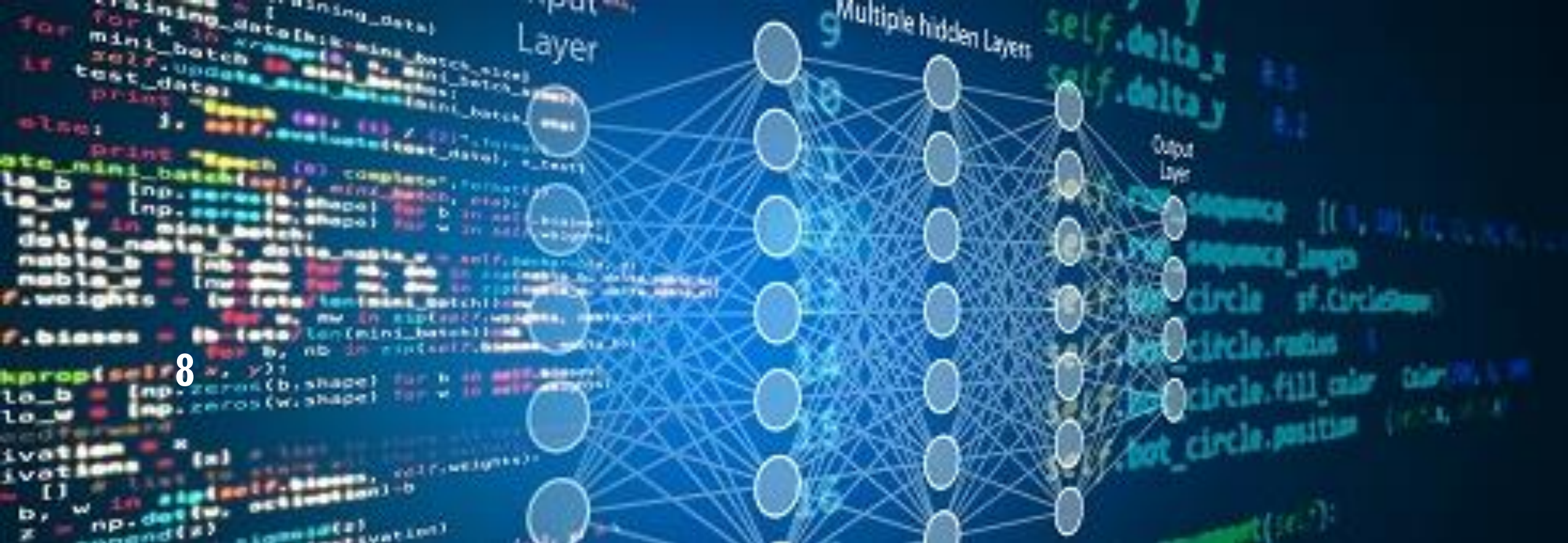
Linear models trained on transformed data often written using Φ ("phi") instead of X :

$$\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2$$
$$\hat{\mathbf{Y}} = \mathbf{X}\theta$$



$$\hat{y} = \theta_0 + \theta_1 \phi_1 + \theta_2 \phi_2$$
$$\hat{\mathbf{Y}} = \Phi\theta$$

Shorthand for "the design matrix after feature engineering"



ONE-HOT ENCODING

REGRESSION USING NON-NUMERIC FEATURES

Look at the tips dataset below

	total_bill	tip	sex	smoker	day
0	16.99	1.01	Female	No	Sun
1	10.34	1.66	Male	No	Sun
2	21.01	3.50	Male	No	Sun
3	23.68	3.31	Male	No	Sun
4	24.59	3.61	Female	No	Sun

In regression, we are bound to only using numeric features in a model to predict the “tip” (e.g., `total_bill`)

By performing feature engineering, we can incorporate **qualitative** features like the day of the week (`day`).



ONE-HOT ENCODING

One-hot encoding is a feature engineering technique to transform **qualitative data** into numeric features for modeling

- Each category of a categorical variable gets its own feature
 - Value = 1 if a row belongs to the category
 - Value = 0 otherwise

Original data		Sunday	Thursday	Friday	Saturday	One-hot encoding
i=1	Sunday	1	0	0	0	
i=2	Sunday	1	0	0	0	
i=3	Thursday	0	1	0	0	
i=4	Friday	0	0	1	0	
i=5	Saturday	0	0	0	1	

REGRESSION USING THE ONE-HOT ENCODING

The one-hot encoded features can then be used in the design matrix to train a model

	total_bill	size	day_Fri	day_Sat	day_Sun	day_Thur
0	16.99	2	0.0	0.0	1.0	0.0
1	10.34	3	0.0	0.0	1.0	0.0
2	21.01	3	0.0	0.0	1.0	0.0
3	23.68	2	0.0	0.0	1.0	0.0
4	24.59	4	0.0	0.0	1.0	0.0

Raw
features

One-hot encoded features

$$\hat{y} = \theta_1(\text{total_bill}) + \theta_2(\text{size}) + \theta_3(\text{day_Fri}) + \theta_4(\text{day_Sat}) + \theta_5(\text{day_Sun}) + \theta_6(\text{day_Thur})$$

Why is the bias column missing? Answered in a few slides!

REGRESSION USING THE ONE-HOT ENCODING

What tip would the model predict for a party with size 3 and a total bill of \$50 eating on a Friday?

$$\hat{y} = \theta_1(\text{total_bill}) + \theta_2(\text{size}) + \theta_3(\text{day_Fri}) + \theta_4(\text{day_Sat}) + \theta_5(\text{day_Sun}) + \theta_6(\text{day_Thur})$$

Model Coefficient	
Feature	
total_bill	0.09
size	0.19
day_Fri	0.75
day_Sat	0.62
day_Sun	0.73
day_Thur	0.67

REGRESSION USING THE ONE-HOT ENCODING

Party of 3, \$50 total bill, eating on a Friday:

$$\hat{y} = \theta_1(\text{total_bill}) + \theta_2(\text{size}) + \theta_3(\text{day_Fri}) + \theta_4(\text{day_Sat}) + \theta_5(\text{day_Sun}) + \theta_6(\text{day_Thur})$$

$$\hat{y} = 0.09(50) + 0.19(3) + 0.75(1) + 0.62(0) + 0.73(0) + 0.67(0)$$

Model Coefficient	
Feature	
total_bill	0.09
size	0.19
day_Fri	0.75
day_Sat	0.62
day_Sun	0.73
day_Thur	0.67

$$\hat{y} = 5.82$$

REGRESSION USING THE ONE-HOT ENCODING

Interpreting the day_Fri coefficient:

$$\hat{y} = \theta_1(\text{total_bill}) + \theta_2(\text{size}) + \theta_3(\text{day_Fri}) + \theta_4(\text{day_Sat}) + \theta_5(\text{day_Sun}) + \theta_6(\text{day_Thur})$$

Model Coefficient	
Feature	
total_bill	0.09
size	0.19
day_Fri	0.75
day_Sat	0.62
day_Sun	0.73
day_Thur	0.67

Interpretation: 0.75 is the predicted tip for a \$0 bill with 0 customers on Friday.

Not a very useful interpretation!

ONE-HOT ENCODE WISELY!

Any set of one-hot encoded columns will always sum to a column of all ones:

Sunday	Thursday	Friday	Saturday	Bias
1	0	0	0	1
1	0	0	0	1
0	1	0	0	1
0	0	1	0	1
0	0	0	1	1

If we also include a bias column in the design matrix, there will be **linear dependence** in the model. $\mathbf{X}^T \mathbf{X}$ is not invertible, and our OLS estimate $\hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y}$ fails.

ONE-HOT ENCODE WISELY!

How to resolve?

1. Designate a one-hot encoded column as a **reference level** and remove the column from X.
2. Do not include a bias column (i.e., **remove the intercept**).
3. Regularize (not part of the course)

Sunday	Thursday	Friday	Saturday	Bias	or	Sunday	Thursday	Friday	Saturday	Bias
1	0	0	0	1		1	0	0	0	1
1	0	0	0	1		1	0	0	0	1
0	1	0	0	1		0	1	0	0	1
0	0	1	0	1		0	0	1	0	1
0	0	0	1	1		0	0	0	1	1

No information is lost. The omitted column is a linear combination of the remaining columns.

ONE-HOT ENCODING WITH OMITTED COLUMN

What happens if we omit the `day_Fri` column and include an intercept?

$$\hat{y} = \theta_0 + \theta_1(\text{total_bill}) + \theta_2(\text{size}) + \theta_3(\text{day_Sat}) + \theta_4(\text{day_Sun}) + \theta_5(\text{day_Thur})$$

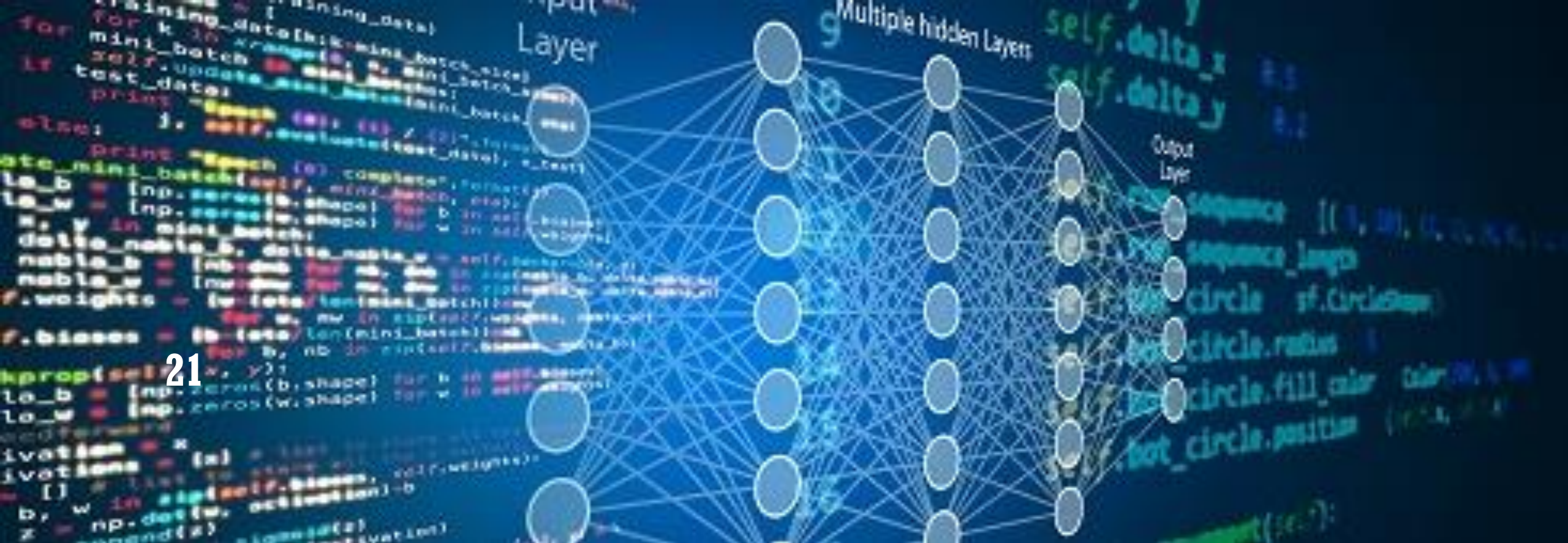
Model Coefficient	
Feature	
Intercept	0.75
total_bill	0.09
size	0.19
day_Sat	-0.12
day_Sun	-0.01
day_Thur	-0.08

Interpretation: For a \$0 bill for a party of size 0, what is the predicted tip for the **reference level** day (i.e., Friday)?

Interpretation: For the same bill and group size, how much larger are predicted Saturday tips, **relative to Friday**?

More useful interpretation than before!

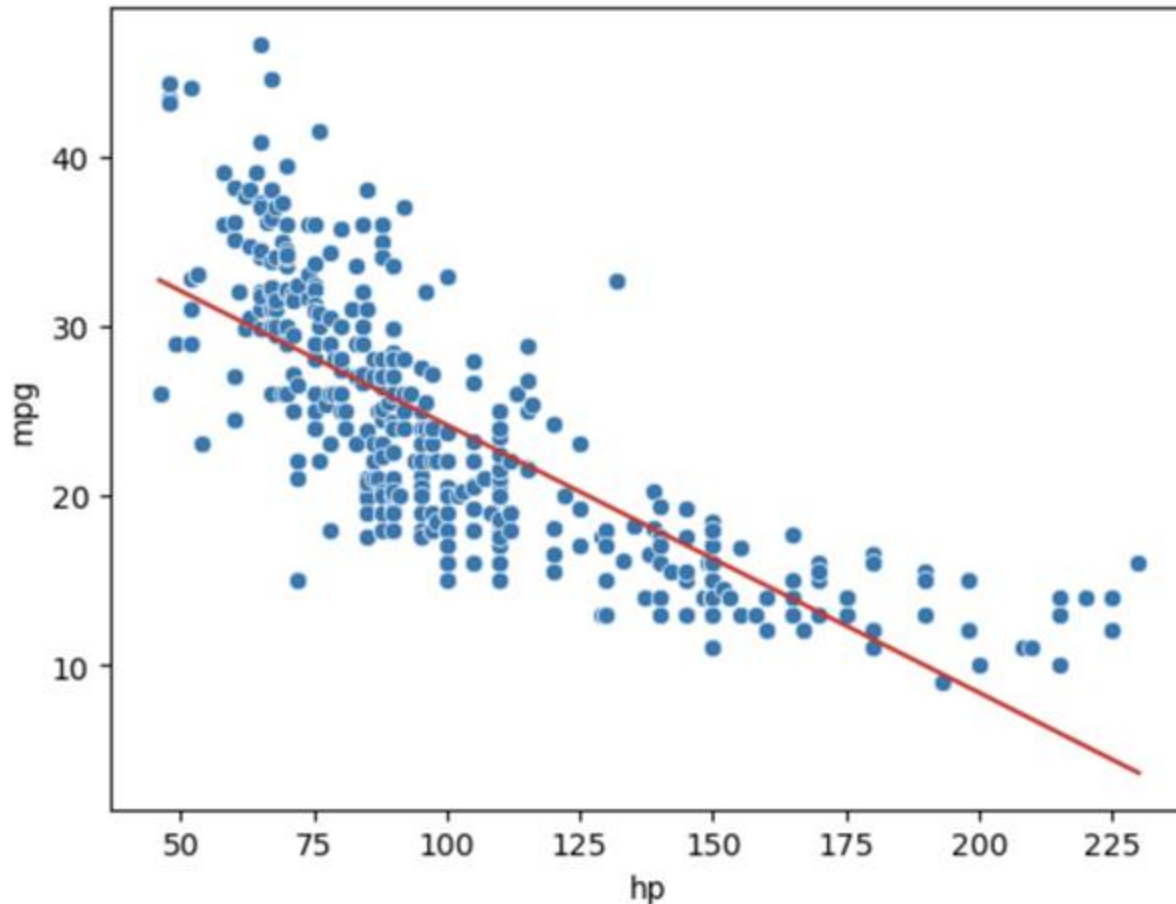
The omitted column approach gives exactly the same predictions as before, but is better for **interpretability**.



POLYNOMIAL FEATURES

ACCOUNTING FOR CURVATURE

We've seen a few cases now where models with linear features have performed poorly on datasets with a clear non-linear curve.



$$\hat{y} = \theta_0 + \theta_1(\text{hp})$$

MSE: 23.94

When our model uses only a single linear feature (**hp**), it cannot capture non-linearity in the relationship

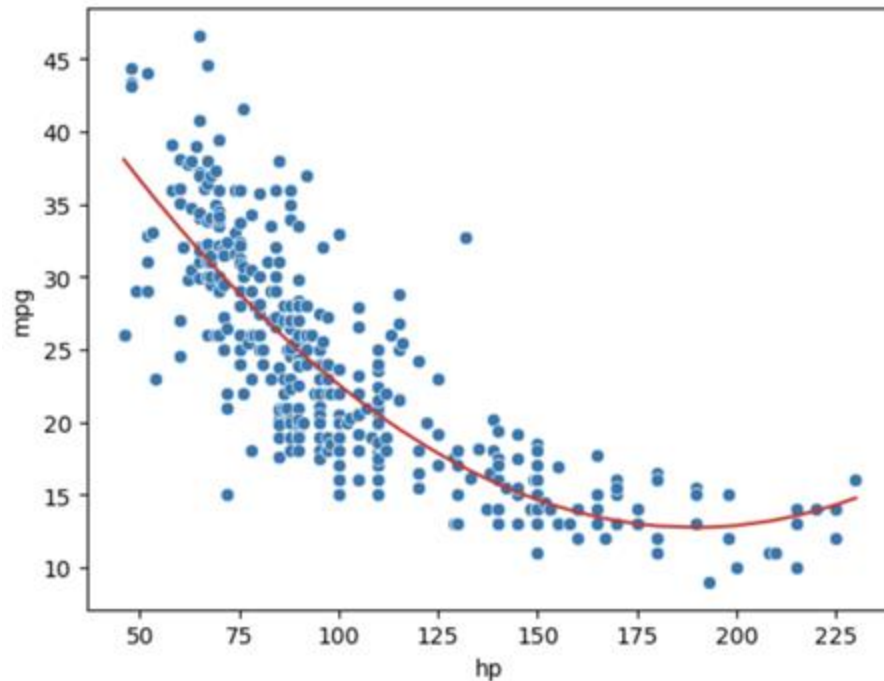
Solution: Incorporate a non-linear feature!

POLYNOMIAL FEATURES

We create a new feature: the square of the hp

$$\hat{y} = \theta_0 + \theta_1(\text{hp}) + \theta_2(\text{hp}^2)$$

This is still a **linear model**. Even though there are non-linear *features*, the model is linear with respect to θ



Degree of model: 2

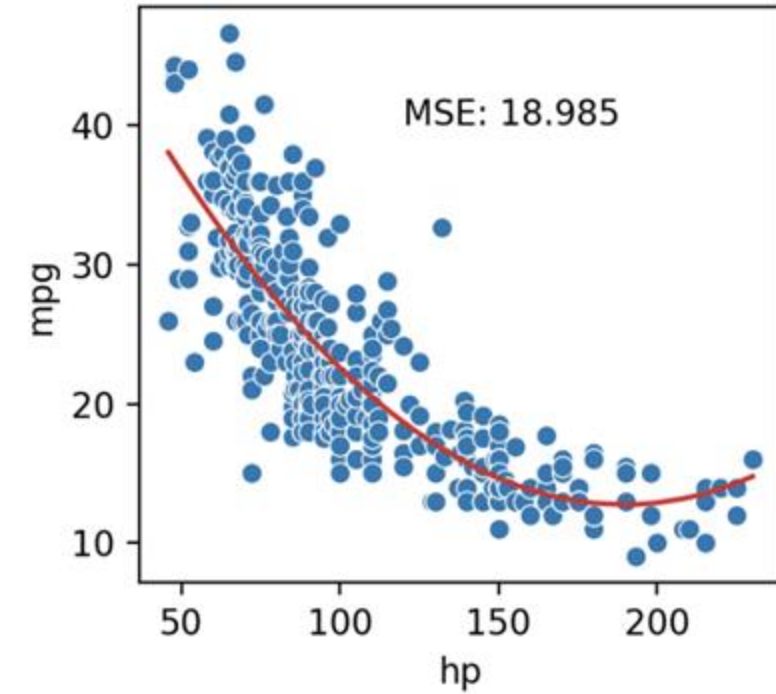
MSE: 18.98

Looking a lot better: Our predictions capture the curvature of the data.

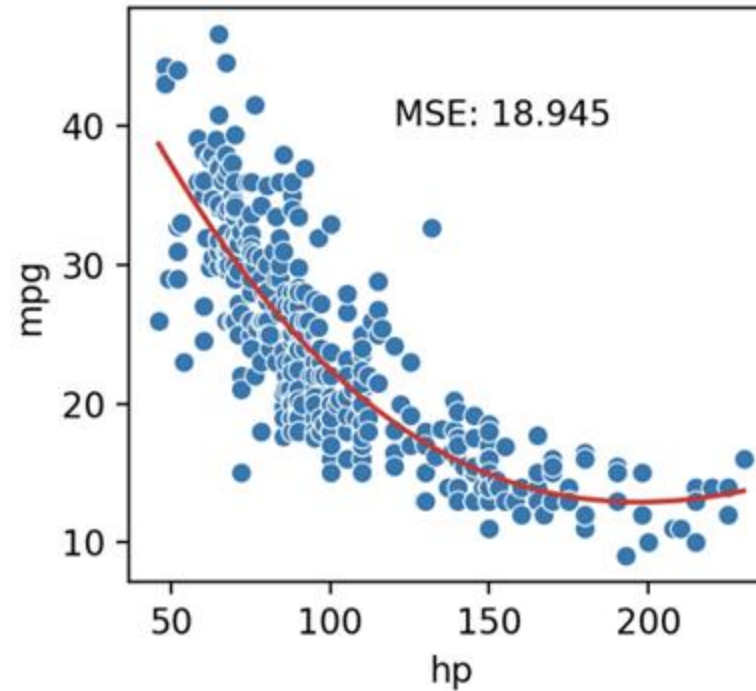
POLYNOMIAL FEATURES

What if we add more polynomial features?

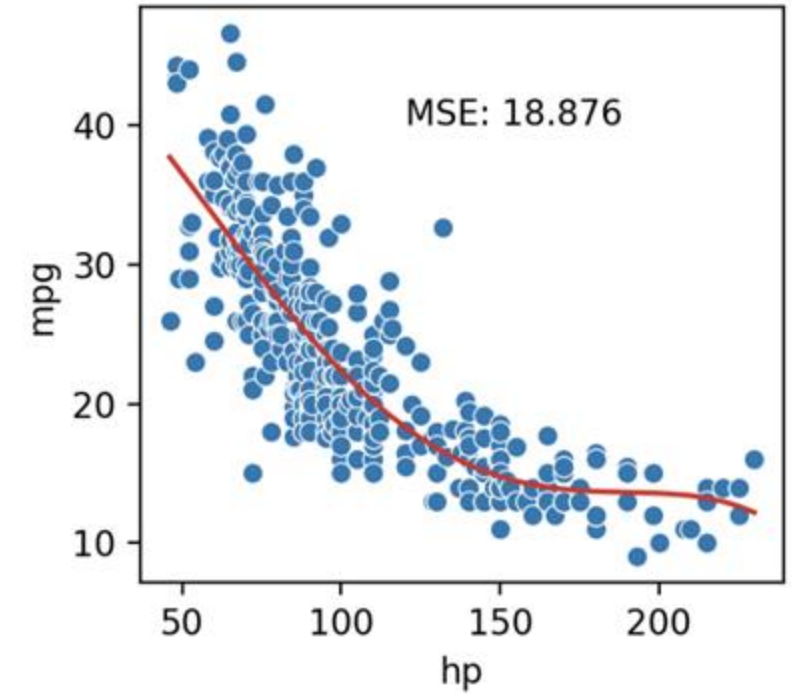
$$\hat{y} = \theta_0 + \theta_1(\text{hp}) + \theta_2(\text{hp}^2)$$



$$\hat{y} = \theta_0 + \theta_1(\text{hp}) + \theta_2(\text{hp}^2) + \theta_3(\text{hp}^3)$$



$$\hat{y} = \theta_0 + \theta_1(\text{hp}) + \theta_2(\text{hp}^2) + \theta_3(\text{hp}^3) + \theta_4(\text{hp}^4)$$

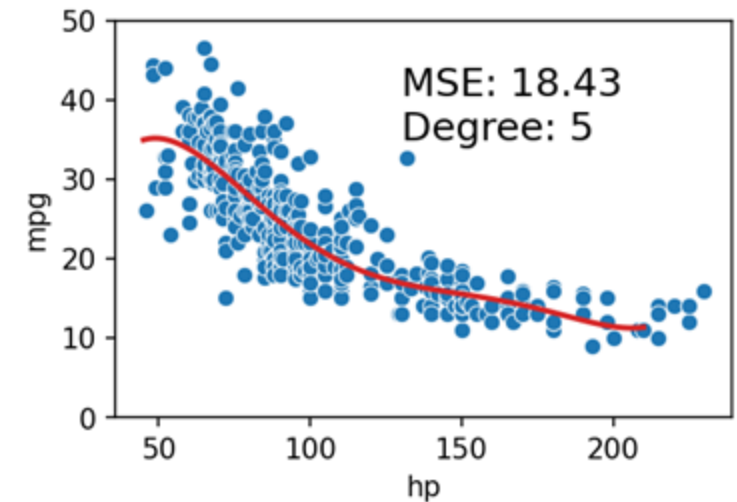
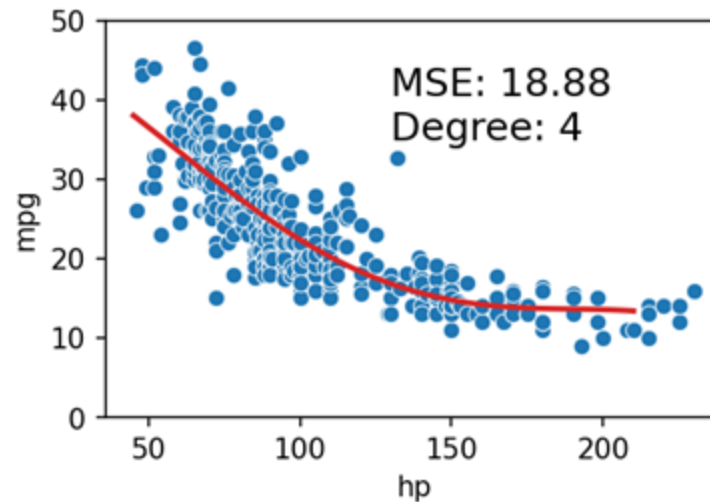
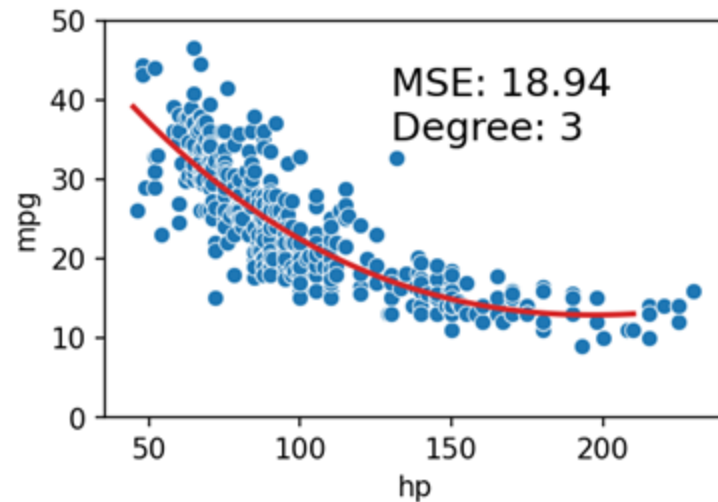
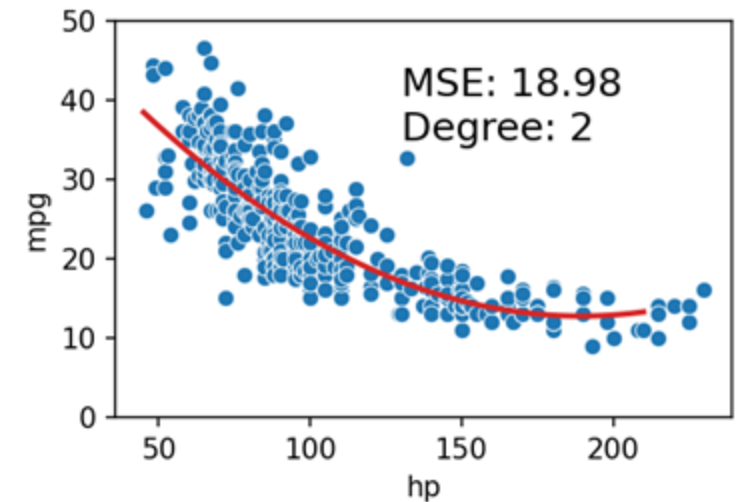
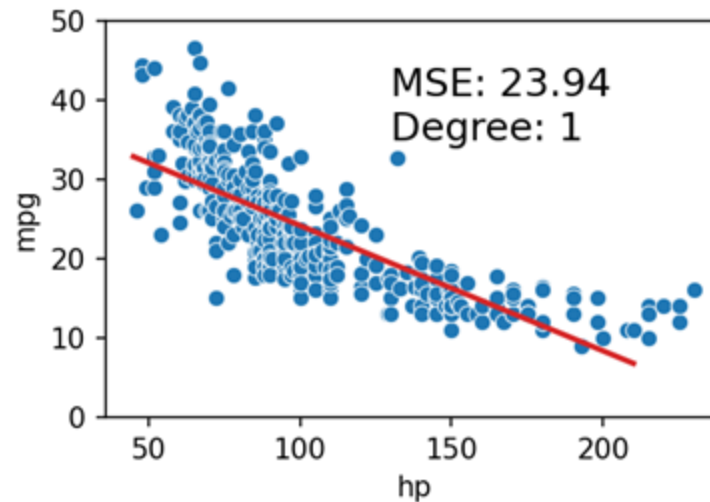
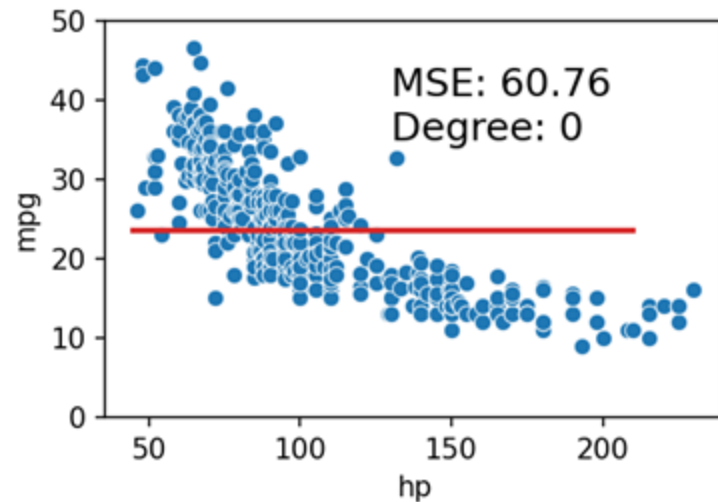


MSE continues to decrease with each additional polynomial term. Should we keep going?



COMPLEXITY & OVERFITTING

HOW FAR CAN WE TAKE THIS?



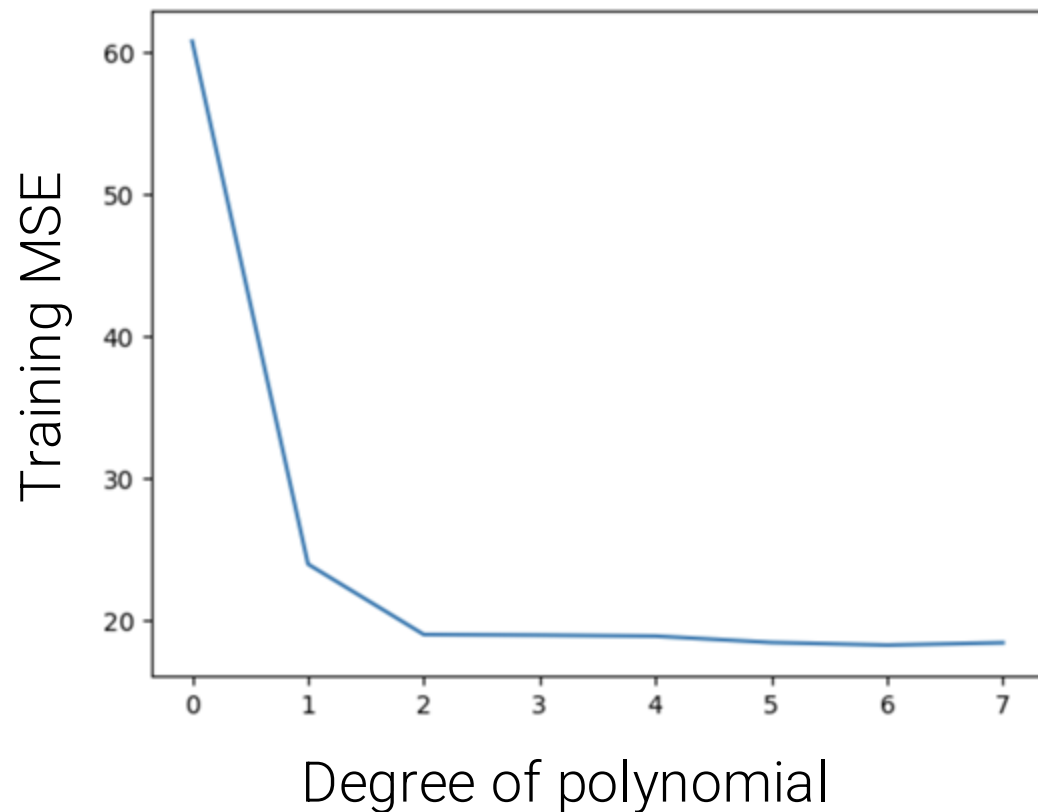
Which higher-order polynomial model fits the best?

MODEL COMPLEXITY

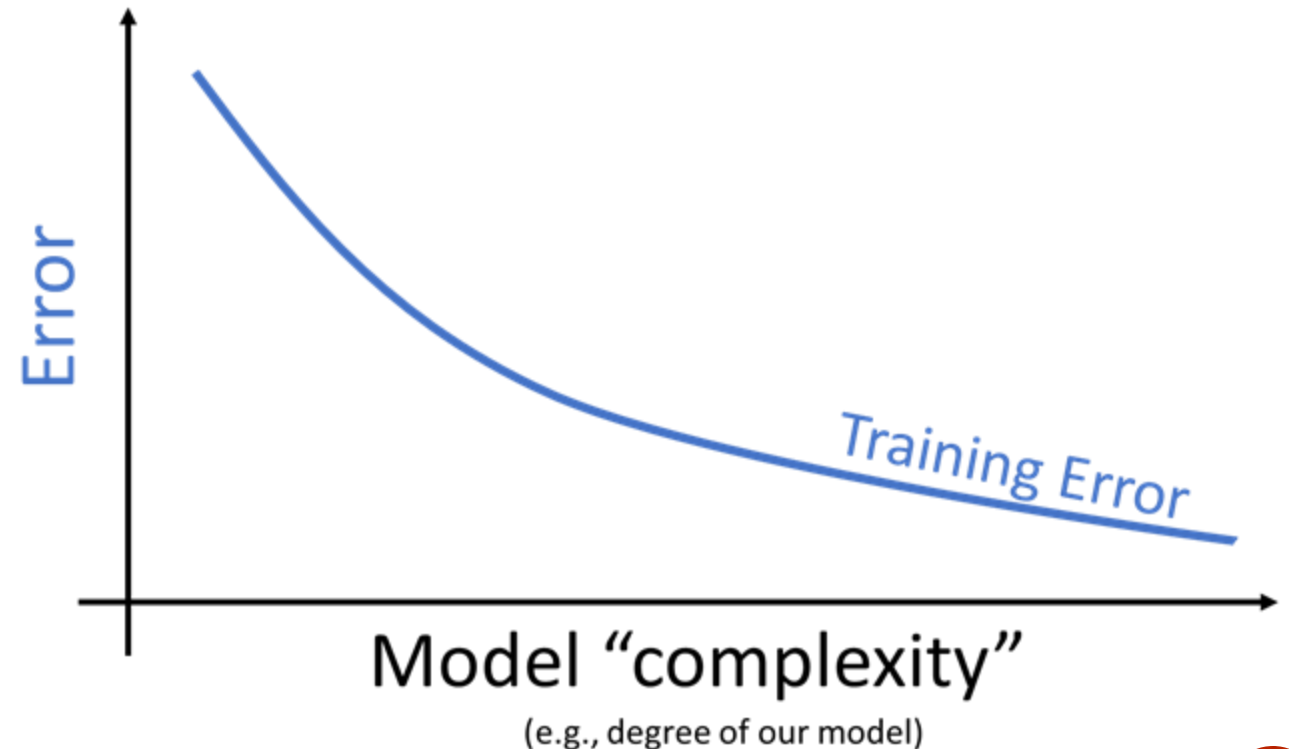
As we continue to add more polynomial features, the MSE decreases.

Equivalently: As the **model complexity** increases, its *training error* decreases.

Our experiment using



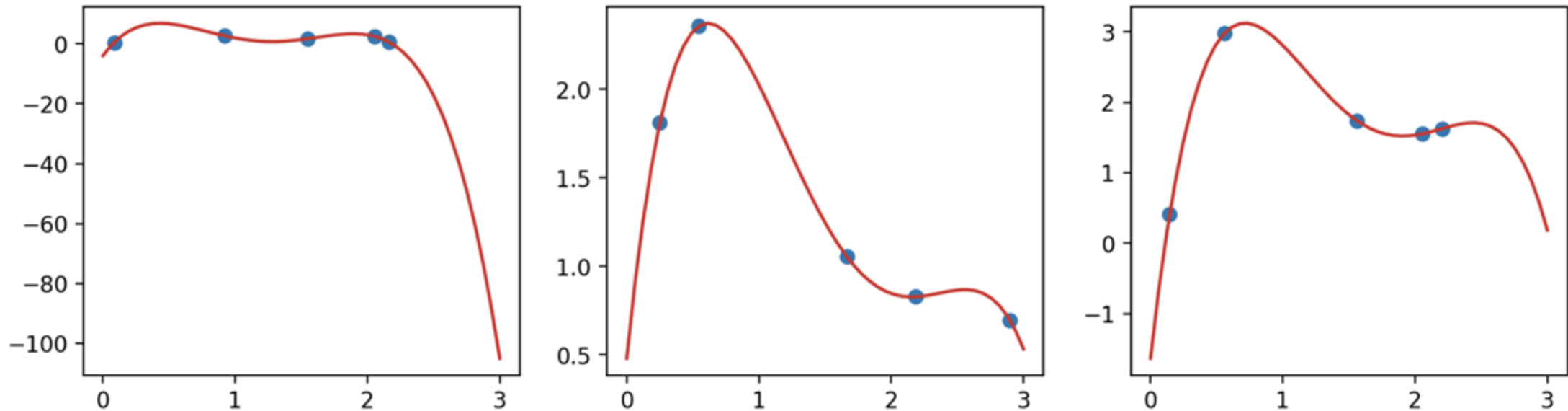
General trend for an arbitrary



AN EXTREME EXAMPLE: PERFECT POLYNOMIAL FITS

Math fact: given N non-overlapping data points, we can always find a polynomial of degree $N-1$ that goes through all those points.

For example, a degree-4 polynomial curve can perfectly model a dataset of 5 datapoints.

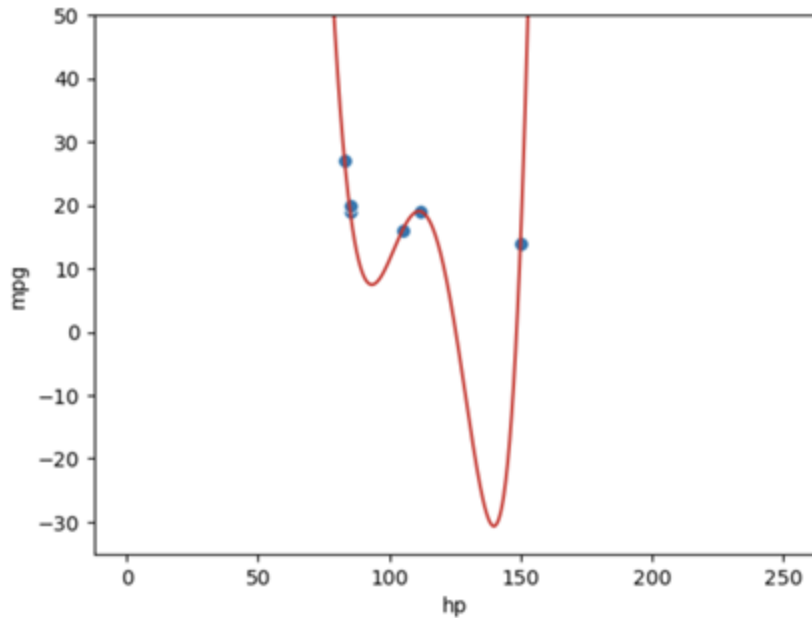


MODEL PERFORMANCE ON UNSEEN DATA

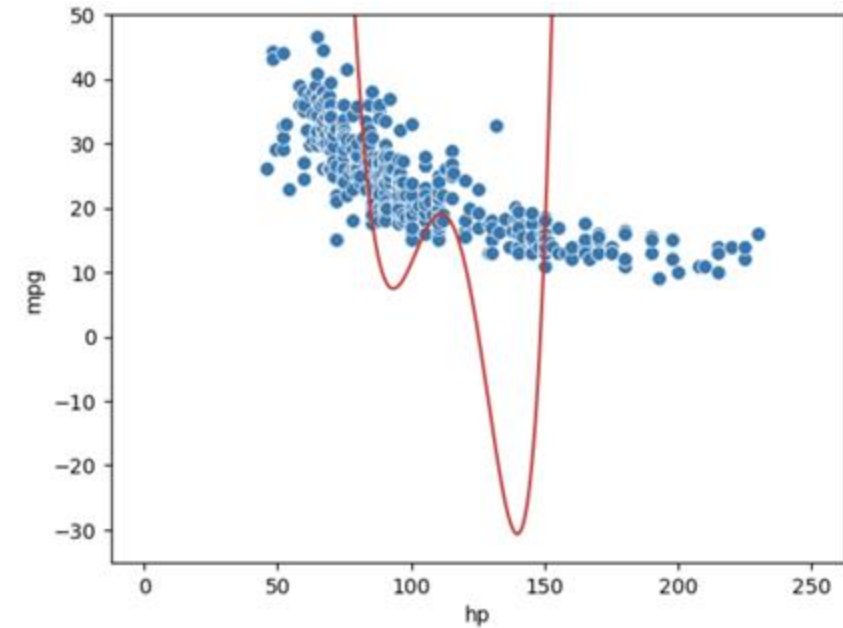
More realistic example:

- We are given a training dataset of just 6 data points.
- We want to train a model to then make predictions on a *different* set of points

We may be tempted to make a highly complex model (e.g., degree 5):



Complex model makes perfect predictions on the training data...



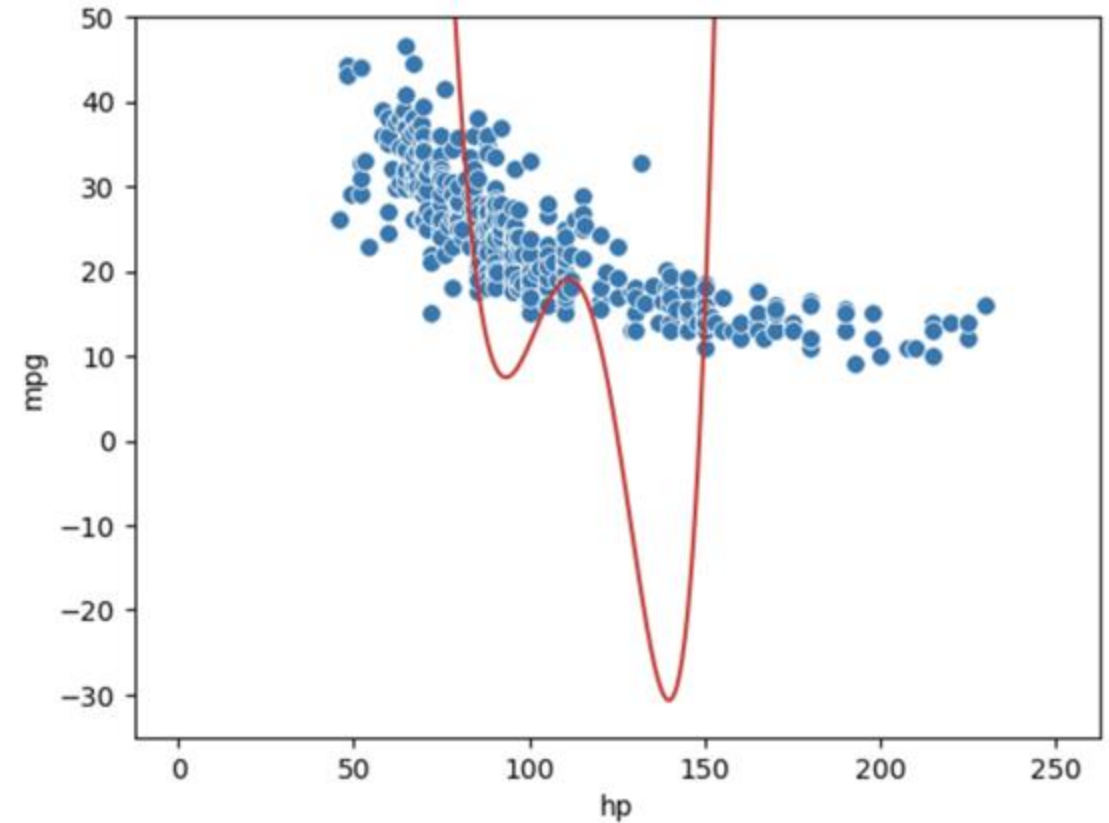
...but performs *horribly* on new data!

MODEL PERFORMANCE ON UNSEEN DATA

What went wrong?

- The complex model **overfit** to the training data – it “memorized” the 6 training points.
- The overfitted model does not **generalize** well to new data.

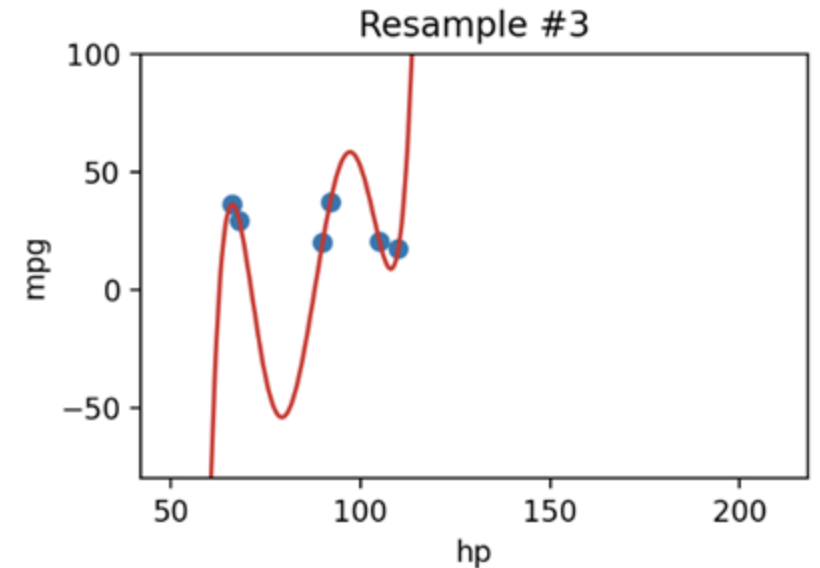
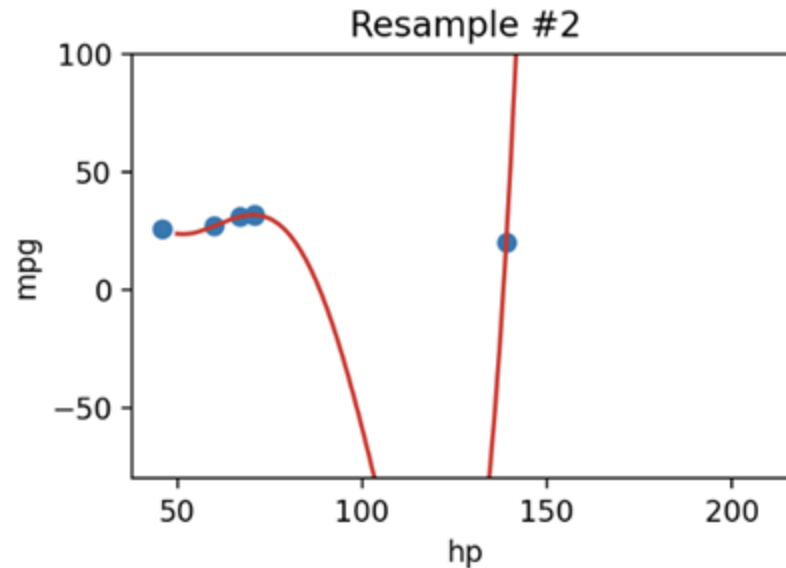
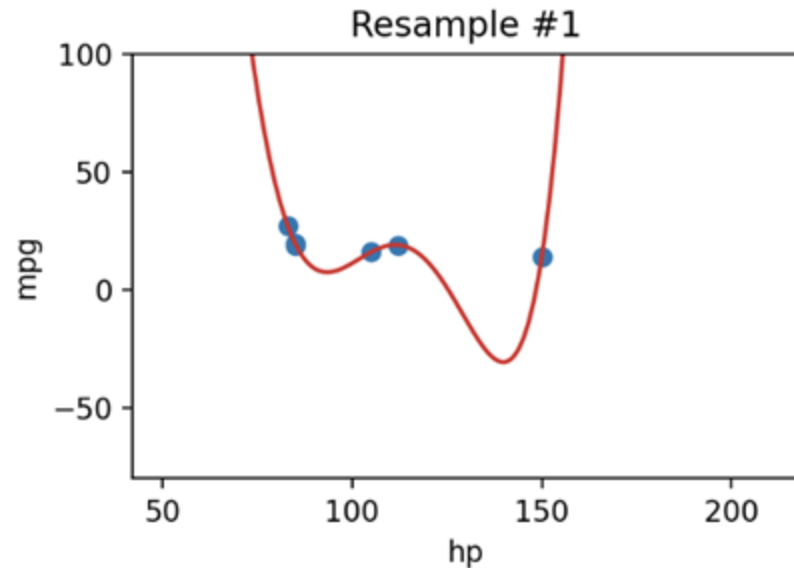
This is a problem: we want models that are generalizable to “unseen” data



MODEL VARIANCE

Complex models are sensitive to the specific dataset used to train them.

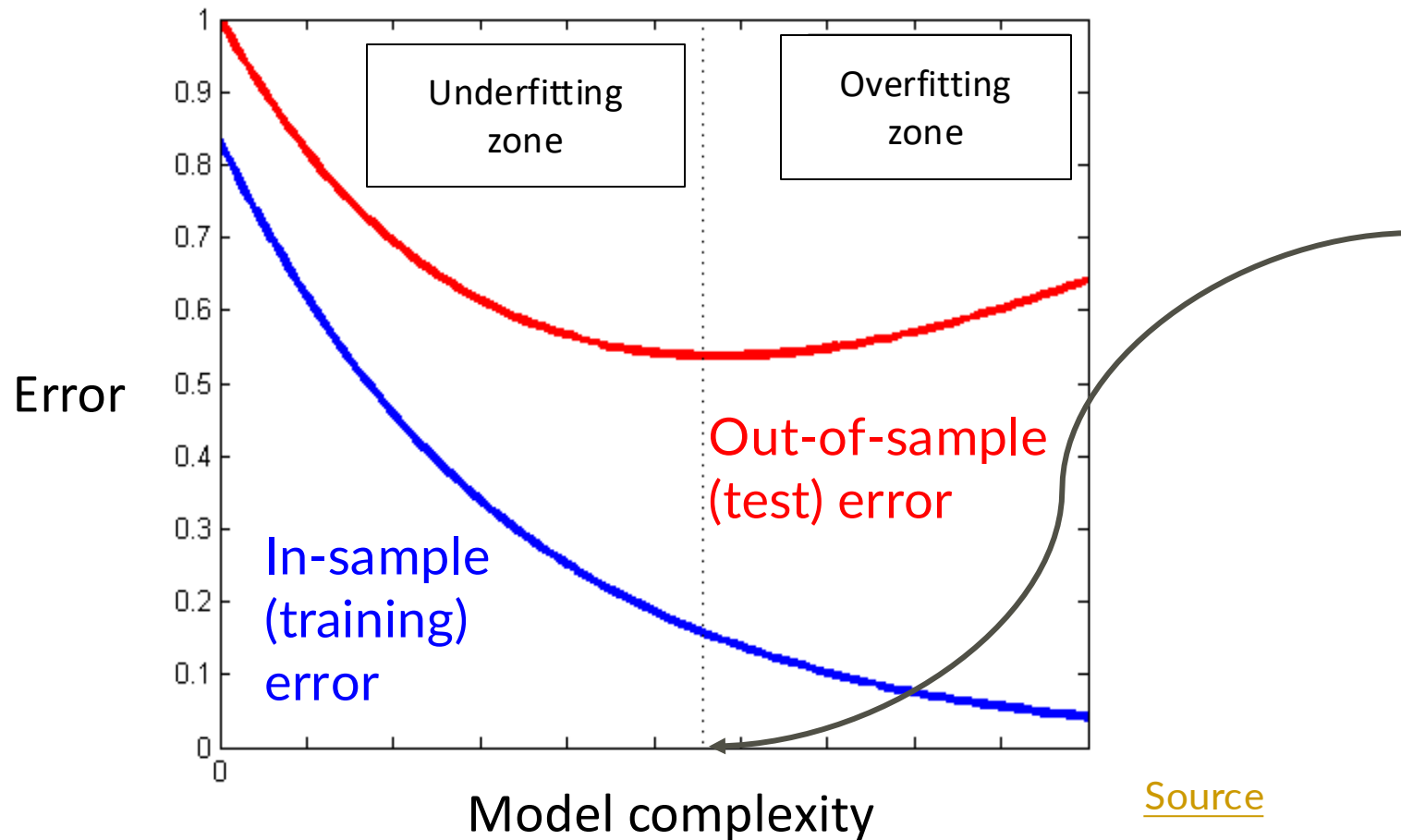
In other words, they have high **variance**, because the fitted model will *vary* a lot across different training samples, even if they were randomly drawn from the same population!



ERROR, VARIANCE, AND COMPLEXITY

We know that we can **decrease training error** by increasing model complexity

However, models that are *too* complex (e.g., high-degree polynomial) do not generalize well.

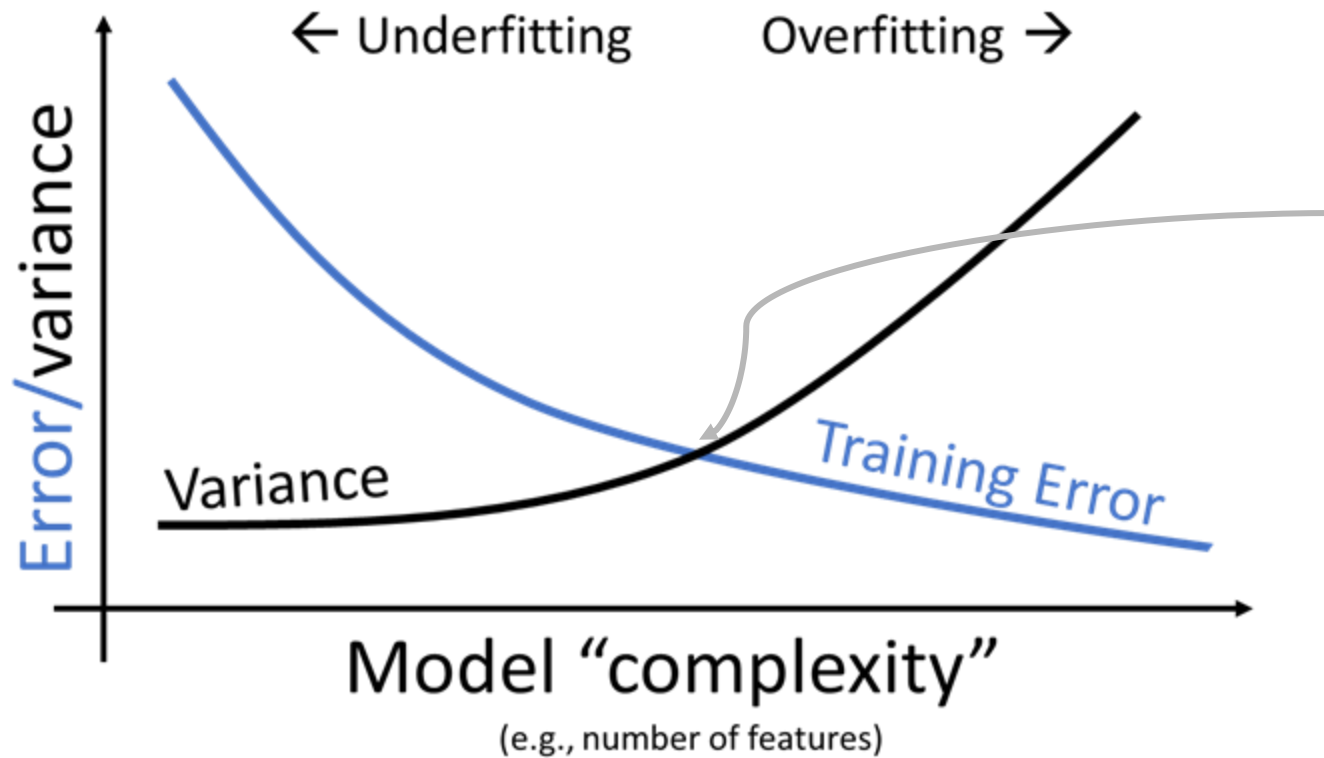


Our goal: Find this “sweet spot”

[Source](#)

ERROR, VARIANCE, AND COMPLEXITY

- The idea of "unseen" data – data that the model did not encounter during training
- The idea of model complexity – a model's complexity influences if it under- or overfits



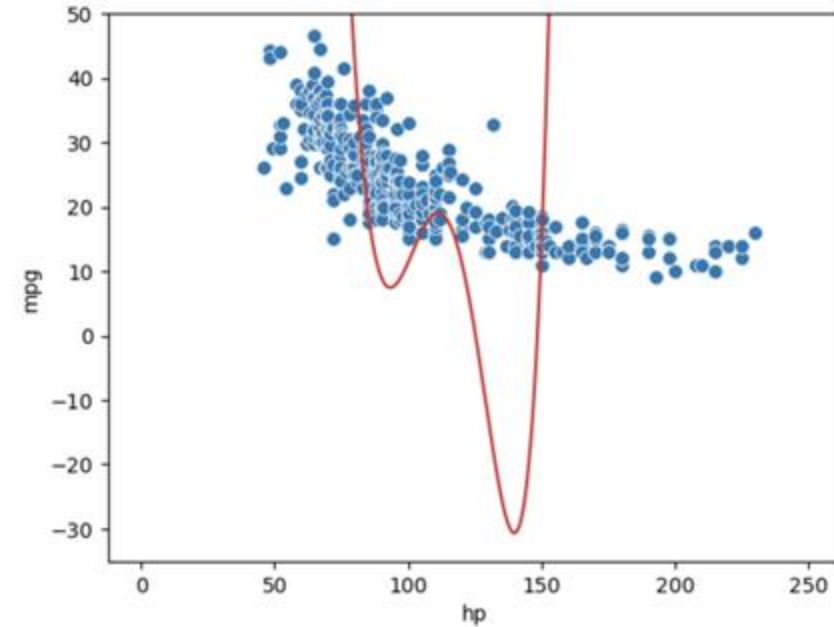
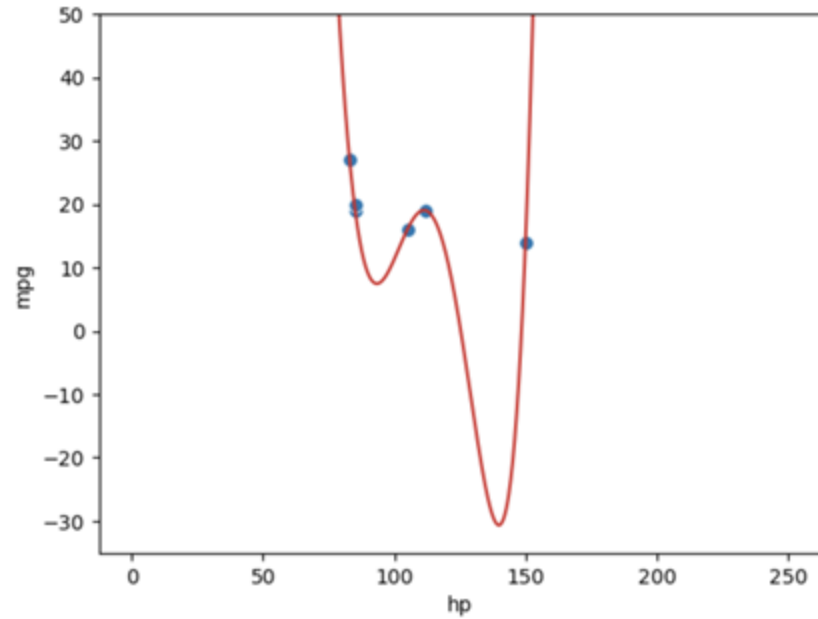
Our goal: find this "sweet spot"



TRAINING, TEST & VALIDATION SETS

MODEL VARIANCE

A complex model may not perform well on data it did not encounter during training.



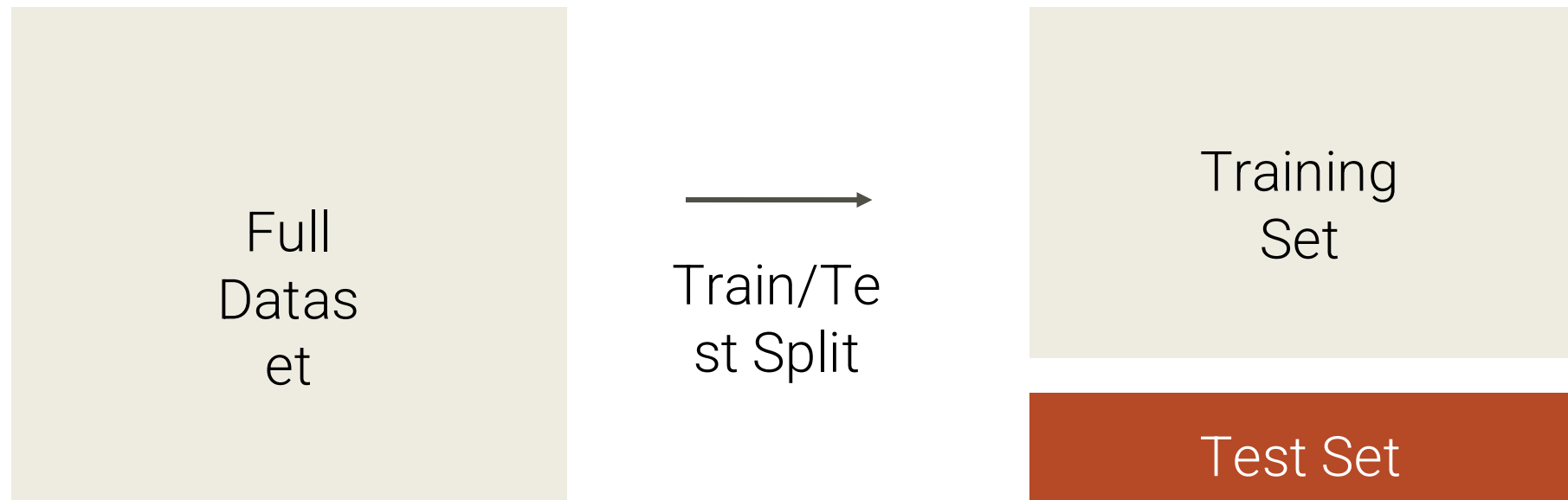
How to quantify performance on this "unseen" data? Introduce a **test set**.

TEST SETS

A **test set** is a portion of our dataset that we set aside for testing purposes.

- We do *not* consider the test set when fitting/training the model.
- The test set is only ever touched once: to compute the performance (MSE, RMSE, etc) of the model *after* all fine-tuning has been completed.

Our new workflow for modeling: First, perform a **train-test split** (see [documentation](#)). Consider only the training set when designing the model. Then, evaluate on the test set.



VALIDATION SETS

What if we were dissatisfied with our test set performance?

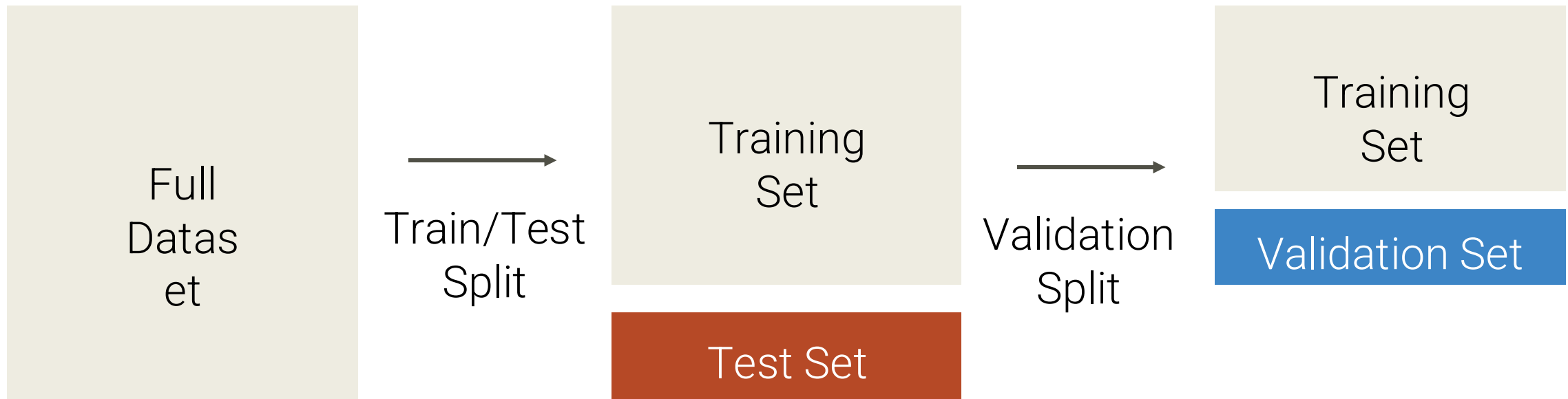
In our current framework, we'd be stuck – we can't then go back and adjust our model, because we'd be *factoring in information from the test set* to design our model. The test set would no longer represent performance on unseen data.

Solution: introduce a **validation set**.

VALIDATION SETS

A **validation set** is a portion of our *training set* that we set aside for assessing model performance while it is *still being developed*.

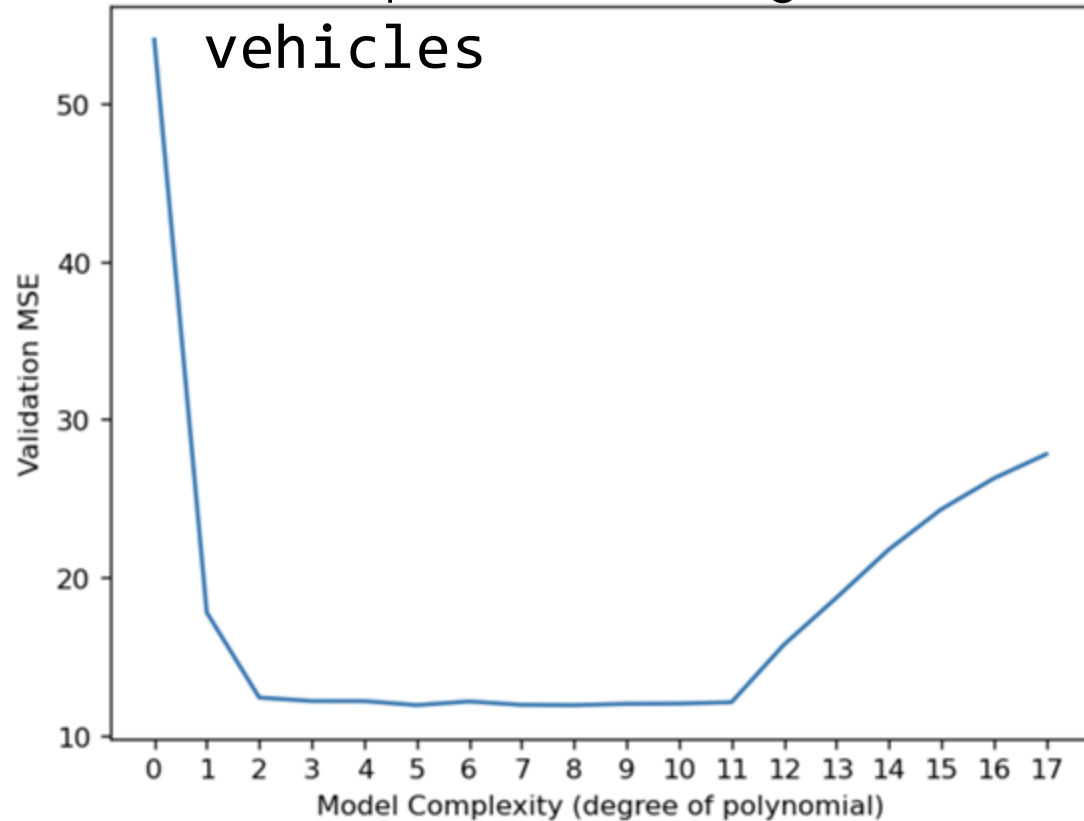
- Train model on the training set. Assess performance on the validation set. Adjust the model, then repeat.
- After *all* model development is complete, assess final performance on the test set.



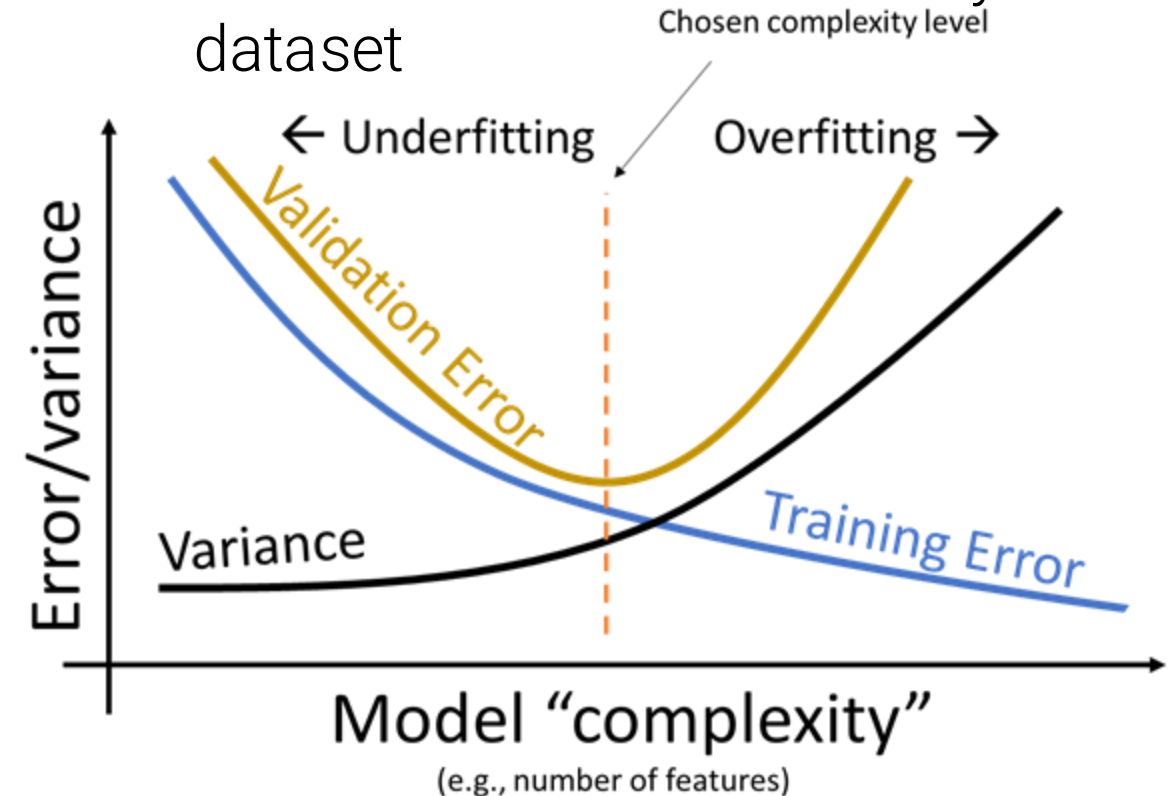
UPDATING OUR UNDERSTANDING OF MODEL COMPLEXITY

Computing the validation error allows us to visualize under- and overfitting.

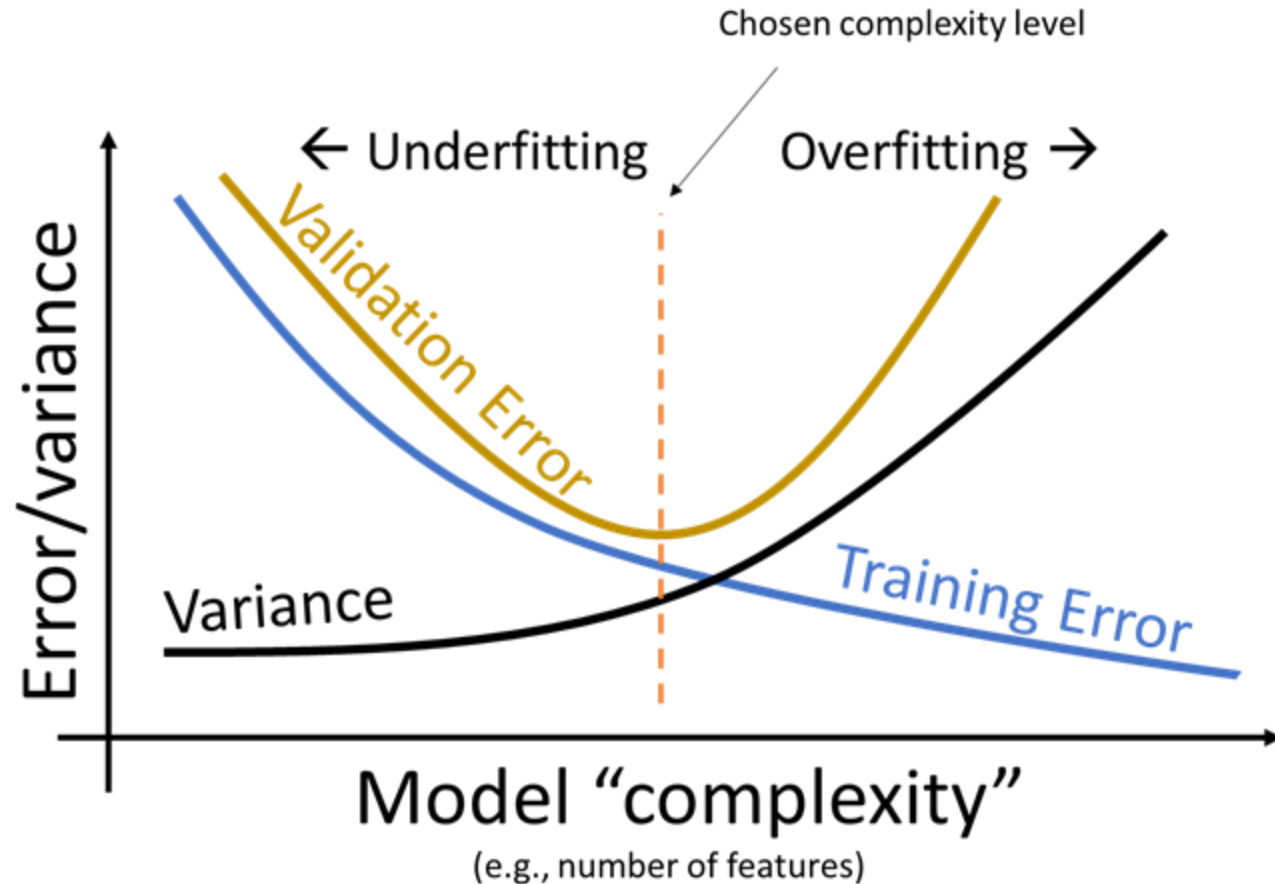
Our experiment using
vehicles



General trend for an arbitrary
dataset



UPDATING OUR UNDERSTANDING OF MODEL COMPLEXITY

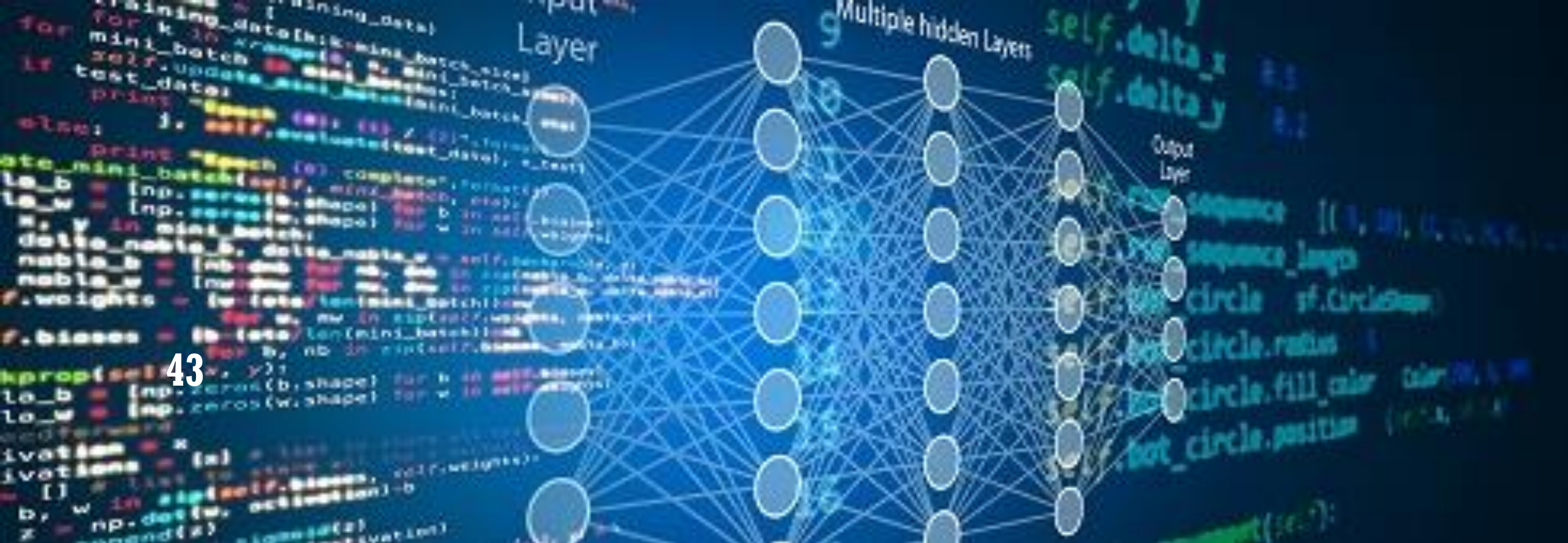


Typically, as model complexity increases:

- Training error decreases
- Variance increases
- Error on validation set decreases, then increases

Our goal: Choose the model complexity that minimizes validation error.

Solution: regularization (out of scope, take ML course)



43

K-FOLD CV

ANOTHER VIEW OF VALIDATION

Introducing a validation set gave us one "extra" chance to assess model performance.

Specifically, now we understand how the model performs on *one* particular set of unseen data.

- It's possible that we may have, by random chance, selected a set of validation points that was *not* representative of other unseen data that the model might encounter.

Val error from train/validation split #1:

14.6104005581132

Val error from train/validation split #2:

24.755706579814404

Val error from train/validation split #3:

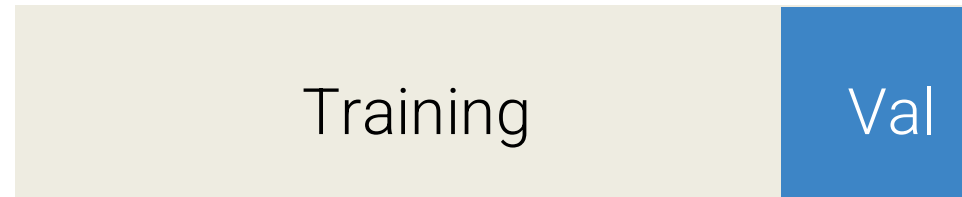
22.23208329959848

Ideally: Assess model performance on *several* different validation sets before touching the test set.

VALIDATION FOLDS

In our original validation split, we set aside $x\%$ of the training data to use for validation.

- For example, 20% of the training data is used for validation



We could have selected *any* 20% portion of the training data for validation.

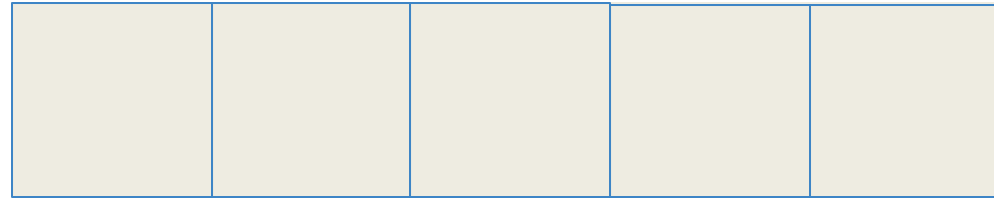


In total, there are 5 non-overlapping “chunks” of datapoints we could set aside for validation.

VALIDATION FOLDS

The common term for one of these chunks is a "fold".

- Our training data has 5 folds, each containing 20% of the datapoints.



Another perspective: we actually have 5 validation sets "hidden" in our training set.

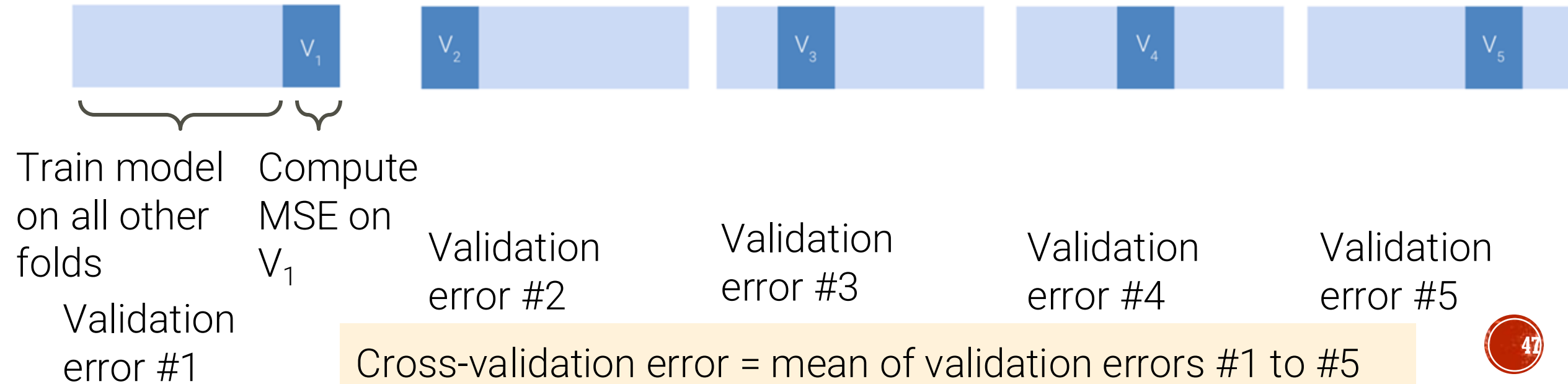
In **cross-validation**, we perform validation splits for *each* of these folds.

K-FOLD CROSS-VALIDATION

For a dataset with K folds:

- Pick one fold to be the validation fold.
- Train model on data from every fold *other* than the validation fold.
- Compute the model's error on the validation fold and record it.
- Repeat for all K folds.

The **cross-validation error** is the average error across all K validation folds.



HYPERPARAMETERS

Cross-validation is often used for **hyperparameter** selection.

Hyperparameter: Value in a model chosen *before* the model is fit to data.

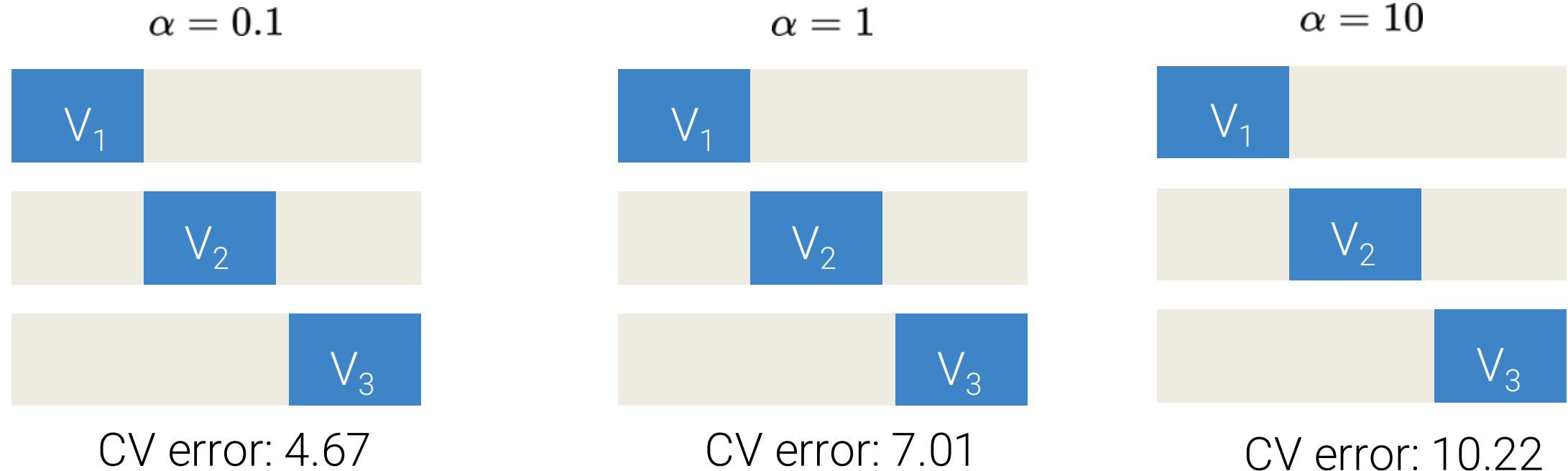
- Cannot solve for hyperparameters via calculus, OLS, gradient descent, etc – we must choose it ourselves.
- Examples
 - Degree of polynomial model
 - Gradient descent learning rate, α
 - Regularization penalty, λ

HYPERPARAMETER TUNING

To select a hyperparameter value via cross-validation:

- List out several different “guesses” for the best hyperparameter.
- For each guess, run cross-validation to compute the CV error for that choice of hyperparameter value.
- Select the hyperparameter value with lowest CV error.

Example: Guesses for learning rate are 0.1, 1, and 10. We decide to apply 3-fold cross-validation.

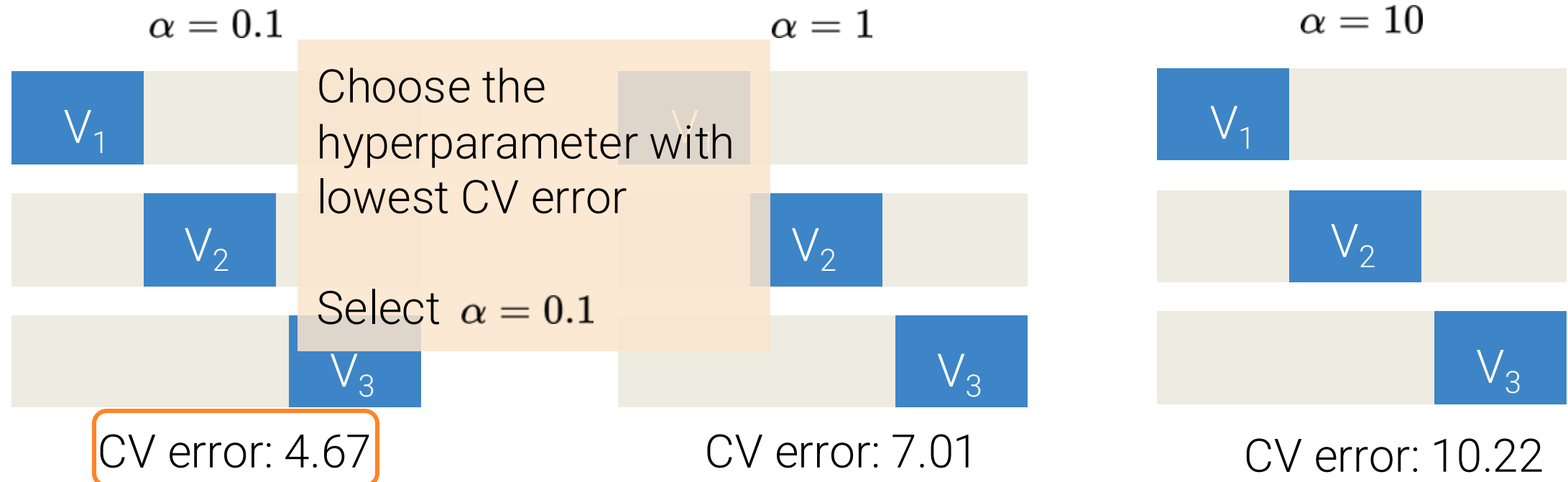


HYPERPARAMETER TUNING

To select a hyperparameter value via cross-validation:

- List out several different “guesses” for the best hyperparameter
- For each guess, run cross-validation to compute the CV error for that choice of hyperparameter value
- Select the hyperparameter value with lowest CV error

Example: Guesses for learning rate are 0.1, 1, and 10. We decide to apply 3-fold cross-validation.



READING MATERIAL

- Chapter# 4: Principle of Data Science
- Chapter# 2 & 4: Hands-On ML [Aurélien]